Georgia State University

CSC 4520/6520
Design & Analysis of Algorithms

CHAPTER 4: DECREASE-AND-CONQUER

(DECREASE-BY-CONSTANT-FACTOR & VARIABLE SIZE DECREASE)

Abdullah Bal, PhD

Objectives

Recap

Generating Permutations

- Minimal Change
- Johnson Trotter

Generating Subsets

- Decrease by one
- Bit Strings

Decrease-by-Constant Factor

Variable-Size-Decrease

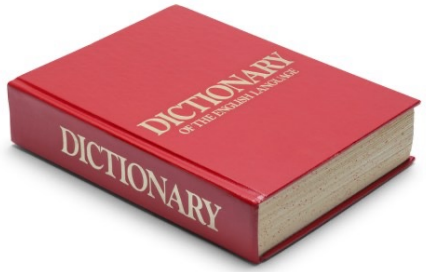
Lomito's Partitioning

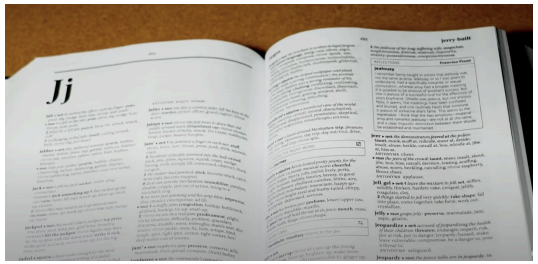
Quickselect

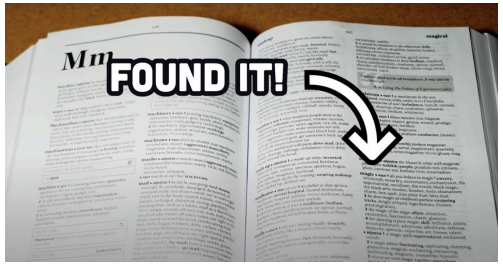
Binary Search Tree

2- Decrease-by-Constant-Factor Algorithms

- In this variation of decrease-and-conquer, instance size is reduced by the same factor (typically, 2)
- Decrease-by-a-constant-factor algorithms usually run in logarithmic time, and being very efficient,
- A reduction by a factor other than two is especially rare





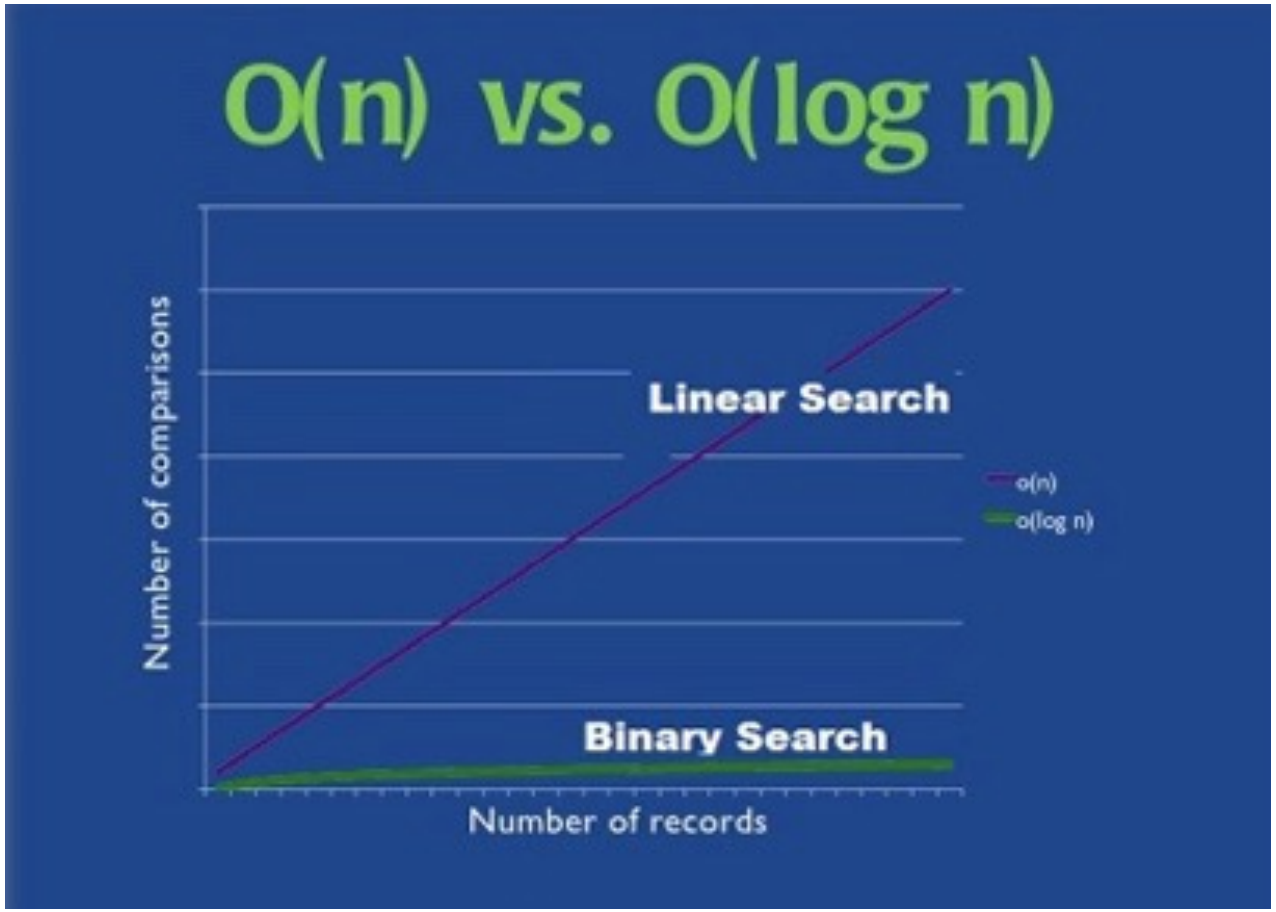


2- Decrease-by-Constant-Factor Algorithms

Examples:

- Binary search and the method of bisection
- Exponentiation by squaring
- Multiplication à la russe (Russian peasant method)
- Fake-coin puzzle

Example 1: Binary Search



Example 1: Binary Search

- Very efficient algorithm for searching in sorted array:

- If $K = A[m]$, stop (successful search);

  otherwise,

- Continue searching by the same method

  in A[0..m-1] if $K < A[m]$

  and

  in A[m+1..n-1] if $K > A[m]$

$$\underbrace{A[0] \ldots A[m-1]}_{\substack{\text{search here if} \\ K < A[m]}} \quad \begin{matrix} K \\ \updownarrow \\ A[m] \end{matrix} \quad \underbrace{A[m+1] \ldots A[n-1]}_{\substack{\text{search here if} \\ K > A[m]}}.$$

Example 1: Binary Search

ALGORITHM *BinarySearch*$(A[0..n-1], K)$

```
//Implements nonrecursive binary search
//Input: An array A[0..n − 1] sorted in ascending order and
//       a search key K
//Output: An index of the array's element that is equal to K
//       or −1 if there is no such element
l ← 0;    r ← n − 1
while l ≤ r do
    m ← ⌊(l + r)/2⌋
    if K = A[m] return m
    else if K < A[m] r ← m − 1
    else l ← m + 1
return −1
```

Example 1: Binary Search

- As an example, let us apply binary search to searching for K = 70 in the array

3	14	27	31	39	42	55	70	74	81	85	93	98

- The iterations of the algorithm are given in the following table:

index	0	1	2	3	4	5	6	7	8	9	10	11	12
value	3	14	27	31	39	42	55	70	74	81	85	93	98
iteration 1	*l*						*m*						*r*
iteration 2								*l*		*m*			*r*
iteration 3								*l*,*m*	*r*				

- Though binary search is clearly based on a recursive idea, it can be easily implemented as a nonrecursive algorithm, too.

Analysis of Binary Search

- The worst-case inputs include all arrays that do not contain a given search key, as well as some successful searches.
- Since after one comparison the algorithm faces the same situation but for an array half the size, we get the following recurrence relation for $C_{worst}(n)$:

$$C_{worst}(n) = C_{worst}(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1, \quad C_{worst}(1) = 1.$$

- Using backward substitution, we obtain

$$C_{worst}(n) = \lfloor \log_2 n \rfloor + 1$$

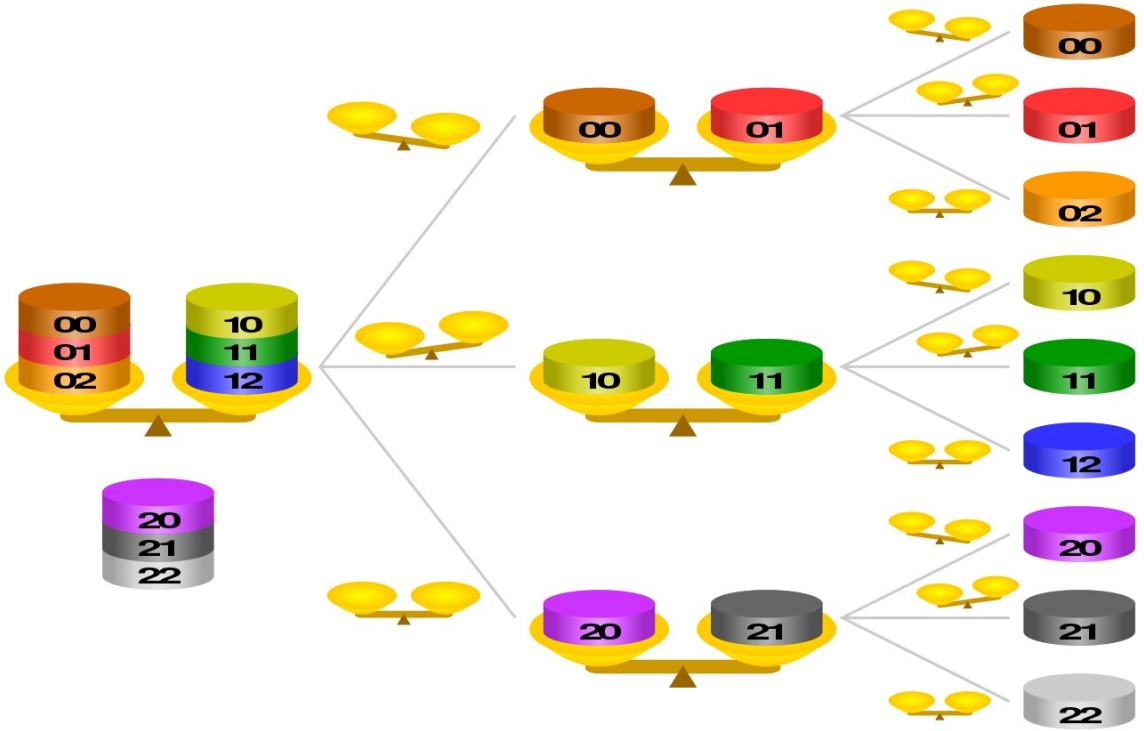
- Time complexity of the binary search Θ(log n).

Analysis of Binary Search

- A sophisticated analysis shows that the average number of key comparisons made by binary search is only slightly smaller than that in the worst case:

$$C_{avg}(n) \approx \log_2 n.$$

Example 2: Fake-Coin Problem



https://en.wikipedia.org/wiki/Balance_puzzle#/media/File:Balance_puzzle_solution_tree_for_9_coins.svg

Example 2: Fake-Coin Problem

- There are *n* identically looking coins one of which is fake.
- There is a balance scale but there are no weights; the scale can tell whether two sets of coins weigh the same and, if not, which of the two sets is heavier (but not by how much).
- Design an efficient algorithm for detecting the fake coin.
- Assume that the fake coin is known to be lighter than the genuine ones.

Example 2: Fake-Coin Problem

- The most natural idea for solving this problem is to divide n coins into two piles of ⌊n/2⌋ coins each, leaving one extra coin aside if n is odd, and put the two piles on the scale.
- If the piles weigh the same, the coin put aside must be fake; otherwise, we can proceed in the same manner with the lighter pile, which must be the one with the fake coin.

Example 2: Fake-Coin Problem

- We can easily set up a recurrence relation for the number of weightings W (n) needed by this algorithm in the worst case:

$$W(n)=W(\lfloor n/2 \rfloor)+1 \text{ for } n>1, \qquad W(1)=0.$$

- This similarity is not really surprising, since both algorithms (binary search and this) are based on the same technique of halving an instance size.
- The solution to the recurrence for the number of weightings is also very similar to the one we had for binary search: $W(n) = \lfloor \log_2 n \rfloor$.

Example 2: Fake-Coin Problem

- It would be more efficient to divide the coins not into two but into *three* piles of about n/3 coins each.
- After weighing two of the piles, we can reduce the instance size by a factor of three.
- Accordingly, we should expect the number of weighing to be about $\log_3 n$, which is smaller than $\log_2 n$.

Example 2: Russian Peasant Multiplication

- The problem: Compute the product of two positive integers

- Can be solved by a decrease-by-half algorithm based on the following formulas.

$$n \cdot m = \frac{n}{2} \cdot 2m$$

  - If n is odd, we need only a slight adjustment of this formula:

$$n \cdot m = \frac{n-1}{2} \cdot 2m + m$$

  - Using these formulas and the trivial case of 1 . m = m to stop
  - We can compute product n . m either recursively or iteratively.

Example 2: Russian Peasant Multiplication

- The algorithm involves just the simple operations of halving, doubling, and adding—a feature that might be attractive, for example, to those who do not want to memorize the table of multiplications.
- It also leads to very fast hardware implementation since doubling and halving of binary numbers can be performed using shifts, which are among the most basic operations at the machine level.
- Compute 50*65

n	*m*	
50	65	
25	130	130
12	260	
6	520	
3	1040	1040
1	2080	2080
		3250

Note: Method reduces to adding *m*'s values corresponding to odd *n*'s.

3- Variable-Size-Decrease Algorithms

- Instance size reduction varies from one iteration to another
- Examples:
  - Euclid's algorithm for greatest common divisor
  - Partition-based algorithm for selection problem
  - Interpolation search
  - Some algorithms on binary search trees
  - Nim and Nim-like games

Example 1: Euclid's Algorithm

- Euclid's algorithm is based on repeated application of equality

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

Ex.: gcd(80,44) = gcd(44,36) = gcd(36, 12) = gcd(12,0) = 12

- One can prove that the size, measured by the second number,decreases at least by half after two consecutive iterations.
- Hence, $T(n) \in \Theta(\log n)$

Example 2: Selection Problem

- Finding the k^{th} smallest element in a list of n numbers.
- This number is called the k^{th} ***order statistic***.
- For k = 1 or k = n, we can simply scan the list in question to find the smallest or largest element, respectively.
- A more interesting case of this problem is for k = ⌈n/2⌉, which asks to find an element that is not larger than one half of the list's elements and not smaller than the other half.
- This middle value is called the ***median***, and it is one of the most important notions in mathematical statistics.

Example 2: Selection Problem

- We can find the k^{th} smallest element in a list by sorting the list first and then selecting the k^{th} element in the output of a sorting algorithm.
- The time of such an algorithm is determined by the efficiency of the sorting algorithm used.
- Thus, with a fast-sorting algorithm such as mergesort, the algorithm's efficiency is in $\Theta(n \log n)$.

Example 2: Selection Problem

median: $k = \lceil n/2 \rceil$

Example: 4, 1, 10, 9, 7, 12, 8, 2, 15 median = ?

- The median is used in statistics as a measure of an average value of a sample.
- In fact, it is a better (more robust) indicator than the mean, which is used for the same purpose.

Partitioning Algorithms for the Selection Problem

- Sorting the entire list is most likely overkill since the problem asks not to order the entire list but just to find its k^{th} smallest element.
- We can take advantage of the idea of ***partitioning*** a given list around some value p of, say, its first element.
- In general, this is a rearrangement of the list's elements so that the left part contains all the elements smaller than or equal to p, followed by the ***pivot*** p itself, followed by all the elements greater than or equal to p.

Two Partitioning Algorithms

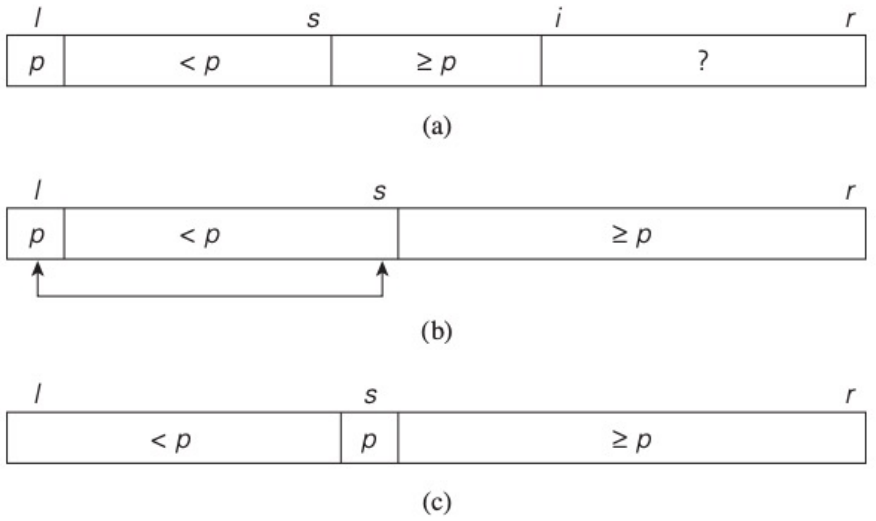
- There are two principal ways to partition an array:
  - One-directional scan (Lomuto's partitioning algorithm)
  - Two-directional scan (Hoare's partitioning algorithm)

Lomuto's Partitioning Algorithm

- It is helpful to think of an array—or, more generally, a subarray under consideration as composed of three contiguous segments.

$$A[l.....r] \qquad (0 \le l \le r \le n - 1)$$

- Listed in the order they follow pivot p, they are as follows:
  - a segment with elements known to be smaller than p,
  - the segment of elements known to be greater than or equal to p,
  - the segment of elements yet to be compared to p.
- Note that the segments can be empty; for example, it is always the case for the first two segments before the algorithm starts.



Lomuto's Partitioning Algorithm

- Starting with i = l + 1, the algorithm scans the subarray A[l….r] left to right, maintaining this structure until a partition is achieved.
- On each iteration, it compares the first element in the unknown segment (pointed to by the scanning index i) with the pivot p.
- If A[i] ≥ p, i is simply incremented to expand the segment of the elements greater than or equal to p while shrinking the unprocessed segment.
- If A[i] < p, it is the segment of the elements smaller than p that needs to be expanded.
- This is done by incrementing s, the index of the last element in the first segment, swapping A[i] and A[s], and then incrementing i to point to the new first element of the shrunk unprocessed segment.
- After no unprocessed elements remain, the algorithm swaps the pivot with A[s] to achieve a partition being sought.

Lomuto's Partitioning Algorithm

ALGORITHM *LomutoPartition*$(A[l..r])$

//Partitions subarray by Lomuto's algorithm using first element as pivot
//Input: A subarray $A[l..r]$ of array $A[0..n-1]$, defined by its left and right
// indices l and r $(l \le r)$
//Output: Partition of $A[l..r]$ and the new position of the pivot
$p \leftarrow A[l]$
$s \leftarrow l$
for $i \leftarrow l+1$ **to** r **do**
  if $A[i] < p$
    $s \leftarrow s+1$; swap$(A[s], A[i])$
swap$(A[l], A[s])$
return s

Tracing Lomuto's Partioning Algorithm

s	*i*							
4	1	10	8	7	12	9	2	15
	s	*i*						
4	1	10	8	7	12	9	2	15
	s						*i*	
4	1	10	8	7	12	9	2	15
		s						*i*
4	1	2	8	7	12	9	10	15
		s						
4	1	2	8	7	12	9	10	15
2	1	4	8	7	12	9	10	15

$p \leftarrow A[l]$
$s \leftarrow l$
for $i \leftarrow l + 1$ **to** r **do**
    if $A[i] < p$
        $s \leftarrow s + 1$; swap($A[s]$, $A[i]$)
swap($A[l]$, $A[s]$)
return s

Quickselect (Partition-based Algorithm)

- Let us assume that the list is implemented as an array whose elements are indexed starting with a 0, and let s be the partition's split position, i.e., the index of the array's element occupied by the pivot after partitioning.
- If $s = k - 1$, pivot p itself is obviously the k^{th} smallest element, which solves the problem.
- If $s > k - 1$, the k^{th} smallest element in the entire array can be found as the k^{th} smallest element in the left part of the partitioned array.
- And if $s < k - 1$, it can be found as the $(k - s)$th smallest element in its right part.
- Thus, if we do not solve the problem outright, we reduce its instance to a smaller one, which can be solved by the same approach, i.e., recursively.
- This algorithm is called ***quickselect***.

Example 1: Tracing Quickselect

- Apply the partition-based algorithm to find the median of the following list of nine numbers: 4, 1, 10, 8, 7, 12, 9, 2, 15.
- Find the 5th (k = [9/2] = 5) smallest element in the array.

0	1	2	3	4	5	6	7	8
s	*i*							
4	1	10	8	7	12	9	2	15
	s	*i*						
4	1	10	8	7	12	9	2	15
	s						*i*	
4	1	10	8	7	12	9	2	15
		s					*i*	
4	1	2	8	7	12	9	10	15
		s						*i*
4	1	2	8	7	12	9	10	15
2	1	**4**	8	7	12	9	10	15

Example 1:Tracing Quickselect

- Since s = 2 is smaller than k − 1 = 4, we proceed with the right part of the array:

0	1	2	3	4	5	6	7	8
			s	*i*				
			8	7	12	9	10	15
				s	*i*			
			8	7	12	9	10	15
				s				*i*
			8	7	12	9	10	15
			7	**8**	12	9	10	15

- Now s = k − 1 = 4, and hence we can stop.
- The found median is 8, which is greater than 2, 1, 4, and 7 but smaller than 12, 9, 10, and 15.

Example 2:

- Apply Lomuto`s Partitioning to find the 4th smallest element

5 7 3 6 2 8

Example 2:

- Apply Lomuto`s Partitioning to find the 4th smallest element

  First Partitioning: 2 3 5 6 7 8

  Second Partitioning: 6 7 8

4th smallest element is 6.

Example 3:

- Apply Lomuto`s Partitioning to find the 5th smallest element

4 6 15 1 2 3 9

Example 3:

- Apply Lomuto`s Partitioning to find the 5th smallest element

4 6 15 1 2 3 9

First Partitioning: 3 1 2 4 15 6 9

Second Partitioning: 9 6 15

Third Partitioning: 6 9

5th smallest element is 6.

Quickselect

ALGORITHM *Quickselect*$(A[l..r], k)$

//Solves the selection problem by recursive partition-based algorithm
//Input: Subarray $A[l..r]$ of array $A[0..n-1]$ of orderable elements and
// integer k $(1 \le k \le r - l + 1)$
//Output: The value of the kth smallest element in $A[l..r]$
$s \leftarrow$ *LomutoPartition*$(A[l..r])$ //or another partition algorithm
if $s = k - 1$ **return** $A[s]$
else if $s > l + k - 1$ *Quickselect*$(A[l..s-1], k)$
else *Quickselect*$(A[s+1..r], k - 1 - s)$

Efficiency of Quickselect

- Partitioning an n-element array always requires n − 1 key comparisons.
- If it produces the split that solves the selection problem without requiring more iterations, then for this best case we obtain $C_{best}(n) = n - 1 \in \Theta(n)$.

Efficiency of Quickselect

- Unfortunately, the algorithm can produce an extremely unbalanced partition of a given array, with one part being empty and the other containing n − 1 elements.
- In the worst case, this can happen on each of the n − 1 iterations.

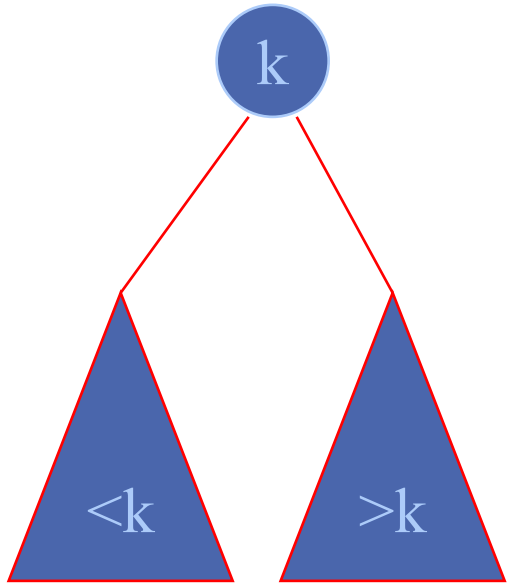
$$C_{worst}(n) = (n - 1) + (n - 2) + \ldots + 1 = (n - 1)n/2 \in \Theta(n^2)$$

which compares poorly with the straightforward sorting-based approach.

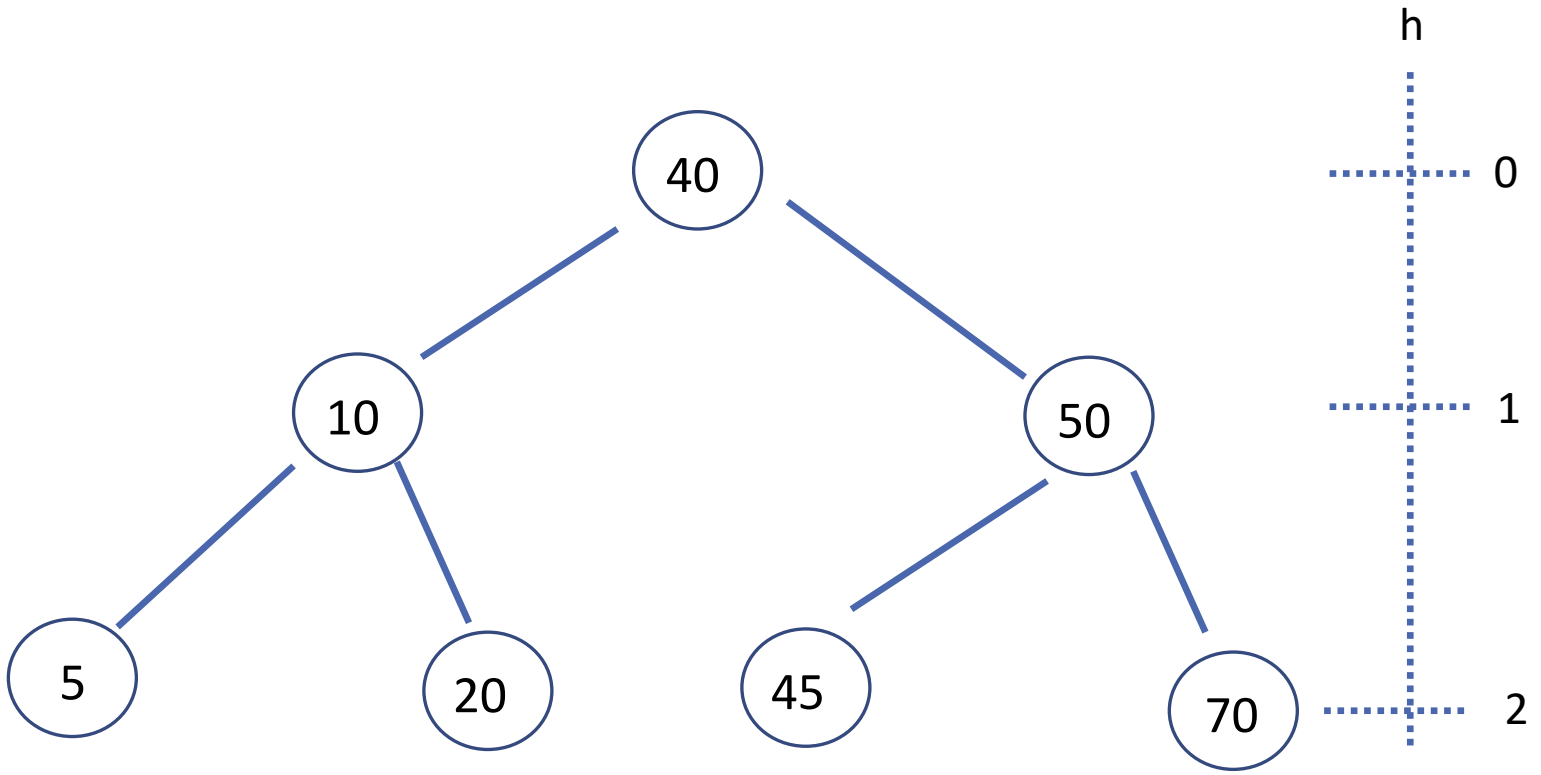
- In fact, computer scientists have discovered a more sophisticated way of choosing a pivot in quickselect that guarantees linear time even in the worst case ($\Theta(n)$) but it is too complicated to be recommended for practical applications.

Binary Search Tree Algorithms

- Several algorithms on BST requires recursive processing of just one of its subtrees, e.g.,
  - Searching
  - Insertion of a new key
  - Finding the smallest (or the largest) key



Binary Search Tree Algorithms



Binary Search Tree Algorithms

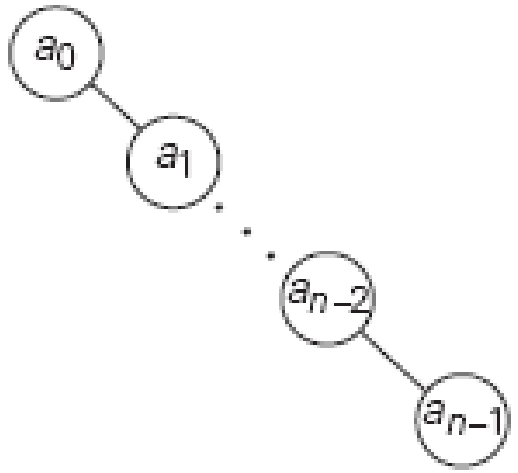
- For every node all elements in the left subtree are smaller and all the elements in the right subtree are greater than the element in the subtree's root.
- If the tree is empty, the search ends in failure.
- If the tree is not empty, we compare v with the tree's root K(r).
- If they match, a desired element is found and the search can be stopped;
- If they do not match, we continue with the search in the left subtree of the root if $v < K(r)$ and in the right subtree if $v > K(r)$.
- Thus, on each iteration of the algorithm, the problem of searching in a binary search tree is reduced to searching in a smaller binary search tree.

Binary Search Tree Algorithms

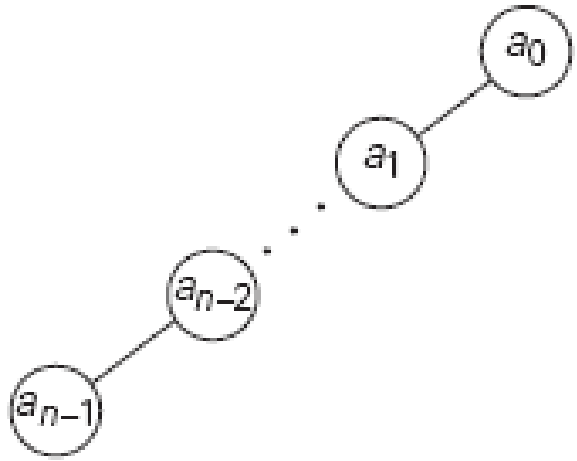
- The most sensible measure of the size of a search tree is its height;
- The decrease in a tree's height normally changes from one iteration to another of the binary tree search—thus giving us an excellent example of a variable-size-decrease algorithm.
- In the worst case of the binary tree search, the tree is severely skewed.
- This happens, in particular, if a tree is constructed by successive insertions of an increasing or decreasing sequence of keys.

Binary Search Tree Algorithms

An increasing sequence of keys



A decreasing sequence of keys



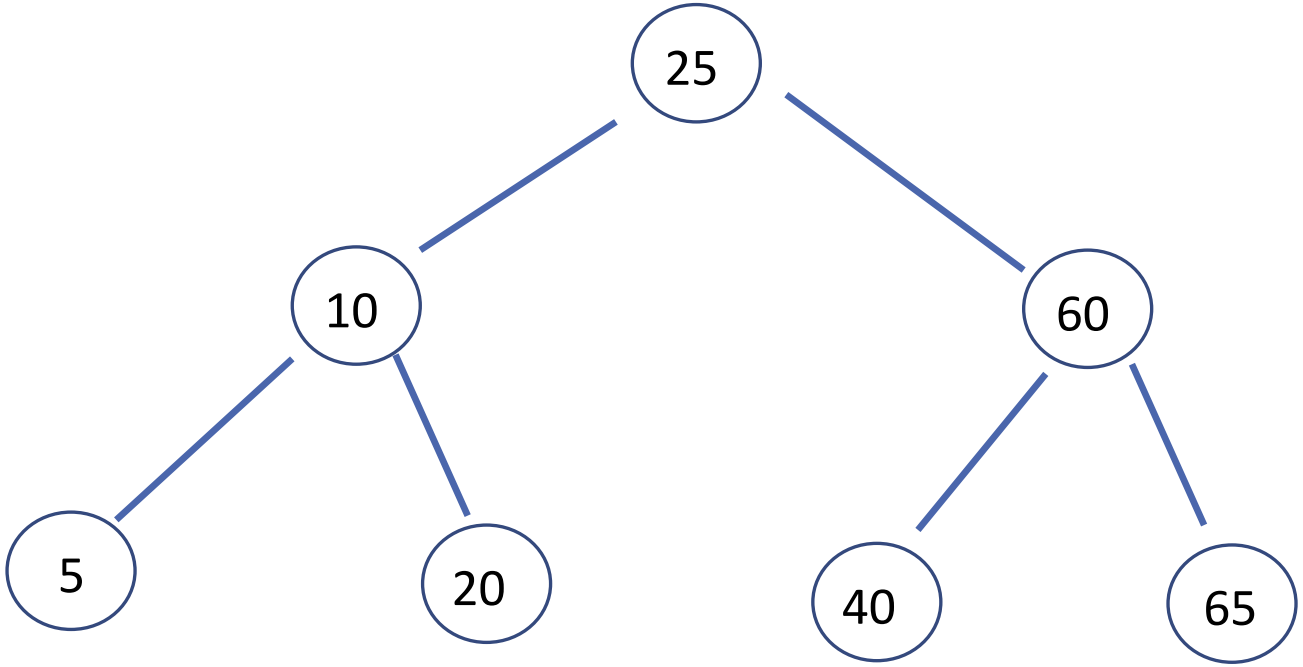
How?

Binary Search Tree Algorithms

Keys: 25, 60, 10, 65, 20, 5, 40

Binary Search Tree Algorithms

Keys: 25, 60, 10, 65, 20, 5, 40

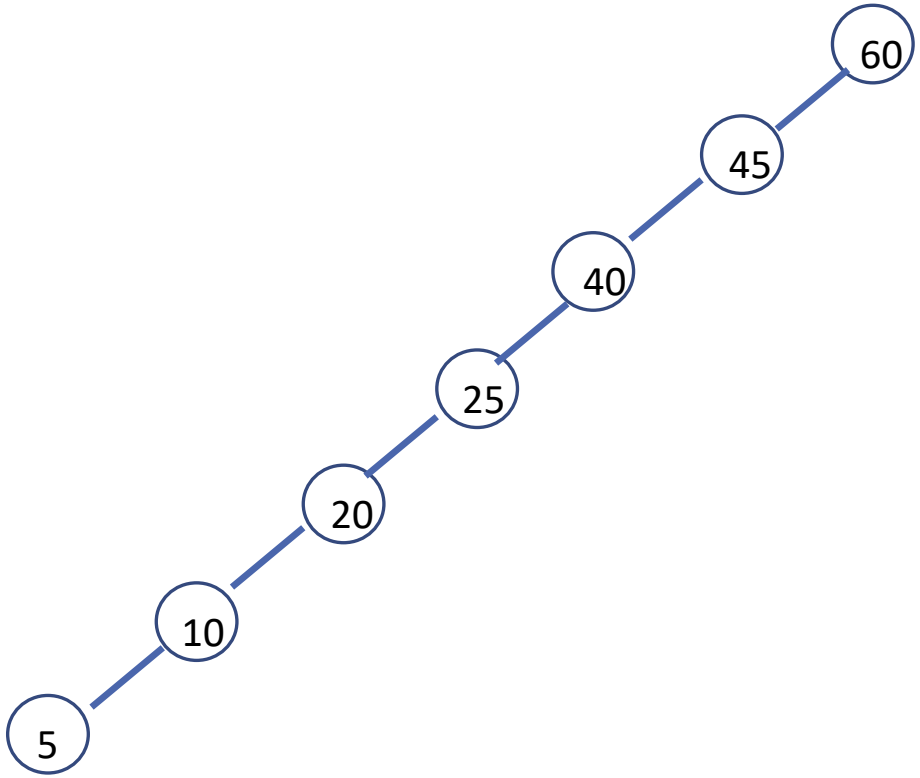


Binary Search Tree Algorithms

Keys: 60, 45, 40, 25, 20, 10, 5

Binary Search Tree Algorithms

Keys: 60, 45, 40, 25, 20, 10, 5



Binary Search Tree Algorithms

- The search for a_{n-1} in such a tree requires n comparisons, making the worst-case efficiency of the search operation fall into Θ(n).
- Fortunately, the average-case efficiency turns out to be in Θ(log n).
- More precisely, the number of key comparisons needed for a search in a binary search tree built from n random keys is about

$$2\ln n \approx 1.39 \log_2 n.$$ for average case.

- Since insertion of a new key into a binary search tree is almost identical to that of searching there, it also exemplifies the variable- size-decrease technique and has the same efficiency characteristics as the search operation.

Summary

- Decrease by constant factor
  - Binary search
  - Fake Coin
  - Russian peasant multiplication
- Variable size decrease
  - Lomuto’s Partitioning
  - Quickselect
  - Binary Search Tree

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition) by Anany Levitin, Pearson.*
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.